

多态

教 师：冀荣华

单 位：人工智能系



面向对象程序设计的基本特点

◆ 抽象

数据抽象：描述某类对象的属性或状态（对象相互区分的物理量）

方法抽象：描述某类对象的共有的行为特征或具有的功能

抽象通过类来实现 封装成一个类

◆ 封装

把对象的属性和行为结合成一个独立的单元，并尽可能隐蔽对象的内部细节。

◆ 继承

1. 一个清楚的边界。对象所有成员都在这个边界内。

◆ 多态

2. 一个或多个接口。外部通过接口访问对象的内部成员

特殊类具有普通类的所有特性和行为
是类之间的一种关系



主要内容

- 一、多态概念
- 二、运算符重载
- 三、虚函数
- 四、虚析构函数
- 五、纯虚函数和抽象类



一、多态概念

多态性是面向对象程序设计的重要特征之一

多态性是指发出同样消息被不同对象接收时有可能导致完全不同行为



多态的实现:

1. 函数重载
2. 运算符重载
3. 虚函数

静态的多态

动态的多态



二、运算符重载

方法一：运算符重载函数作为类的友元函数进行使用

1. 如果需要重载一个运算符，使之能够用于操作某类对象的私有成员，可以此将运算符重载为该类的友元函数
2. 函数的形参代表依自左至右次序排列的各操作数



二、运算符重载

```
#include <iostream.h>
```

```
class Complex
```

```
{ public : Complex(double a=0.0,double b=0.0)
```

```
{ r=a; i=b; }
```

```
void display()
```

```
{ cout<<"Real="<<r<<"\t"<<"Image="<<i<<"\n"; }
```

```
friend Complex operator +(Complex & , Complex & );
```

```
private: double r,i; };
```

```
Complex operator +(Complex & t1, Complex & t2)
```

```
{ double r,i;
```

```
r = t1.r+t2.r; i=t1.i+t2.i;
```

```
Complex result(r,i); return result; }
```

```
void main()
```

```
{ Complex c1(3.2,1.2),c2(1.1,1.1),c3;
```

```
c3=c1+c2;
```

```
c3.display(); }
```

二、运算符重载

方法二：运算符重载函数作为类的成员函数

重载为类成员函数时

参数个数=原操作数个数-1 （后置++、--除外）

双目运算符、单目运算符的重载设计均有所不同



二、运算符重载

```
void main()
```

```
{ Complex c1(3.2,3.2),c2(1.2,1.2),c3;
```

```
    c3=c1+c2;//等价于c3=c1. operator +(c2) ;
```

```
    c3.display();
```

```
}
```



二、运算符重载

```
#include <iostream.h>

class Complex
{ public :Complex(double a=0.0,double b=0.0)
    {   r=a;   i=b;   }
    void display()
    {   cout<<"Real="<<r<<"\t"<<"Image="<<i<<"\n"; }
    Complex operator +(Complex &t )
    {   Complex result(r+t.r,i+t.i);
        return result;
    }

    private:   double r, i ;   };

void main()
{   Complex c1(3.2,1.2),c2(1.1,1.1),c3;
    c3=c1+c2;
    c3.display();
}
```

二、运算符重载

- 运算符重载函数本身就是函数，那么在函数体内部可以做任何事情的
- 从不违背常规思维的角度来说，不要让运算符重载函数来做与其重载的符号意义上完全不相符的工作
- 大多数系统预定义运算符都能重载，除以下：
..、::、*(当乘法运算符时可重载)、sizeof、?:
- 重载时的注意事项：
 - 不能改变优先级；
 - 不能改变结合性；
 - 不能改变运算符所需操作数的个数；
 - 重载后，可按这些运算符的表达方式使用



二、运算符重载

小结

运算符重载是对已有的运算符赋予多重含义

必要性

C++中预定义的运算符其运算对象只能是基本数据类型，而不适用于用户自定义类型（如类）

实现机制

将指定的运算表达式转化为对运算符重载函数的调用，运算对象转化为运算符重载函数的实参。

编译系统对重载运算符的选择，遵循函数重载的选择原则

实现方式

1. 类的友元函数
2. 类的成员函数



二、运算符重载

运算符函数调用形式

习惯形式	友元运算符函数调用形式	成员运算符函数调用形式
$a+b$	$\text{operator}+(a, b)$	$a.\text{operator}+(b)$
$-a$	$\text{operator}-(a)$	$a.\text{operator}-()$
$a++$	$\text{operator}++(a, 0)$	$a.\text{operator}++(0)$



三、虚函数

```
#include <iostream>
#include <string>
using namespace std;
class Student
{public: Student(int n,string nam,float s)
    {num=n;name=nam;score=s;}
    void display()
    {cout<<"num:"<<num<<"\nname:"<<name<<"\nscore:"<<score<<"\n\n";}
protected:int num;          string name;          float score;          };
class Graduate:public Student
{public: Graduate(int n,string nam,float s,float p):Student(n,nam,s),pay(p)
    {}
    void display()
    {cout<<"num:"<<num<<"\nname:"<<name<<"\nscore:"<<score<<"\npay="<<pay<<endl;}
private: float pay;
};
void main()
{ Student stud1(1001,"Li",87.5);
  Graduate grad1(2001,"Wang",98.5,563.5);
  Student *pt=&stud1;
  pt->display();
  pt=&grad1;
  pt->display();
}
```

三、虚函数

联编

把函数名与函数体的程序代码连接在一起的过程

静态联编

在编译阶段完成的联编。

通过匹配实参与形参，确定调用哪个函数。

要求在编译时就知道调用函数的全部信息

静态联编的函数调用速度很快，效率高，但缺乏灵活性；

动态联编：

指在程序执行之前，根据函数名和参数无法确定应该调用哪一个函数，必须在程序的执行过程中，**根据具体的执行情况来动态地确定**



三、虚函数

分析上述问题：

静态联编机制首先将指向基类对象的指针`pt`与基类的成员函数`display()`连接在一起

这样，不管指针`pt`再指向哪个对象，`pt-> display()`调用的总是基类中的成员函数`display()`

若要调用派生类中的成员函数可以采用显式的方法。

`stud1.display();`

或者采用对指针强制类型转换的方法，如：

`((Graduate *)pt)->.display();`



三、虚函数

使用对象指针的目的就是为了表达一种动态的性质，
即当指针指向不同对象时执行不同的操作

解决方案——**虚函数**



三、虚函数

```
#include <iostream>
#include <string>
using namespace std;
class Student
{public: Student(int n,string nam,float s)
    {num=n;name=nam;score=s;}
    virtual void display()
    {cout<<"num:"<<num<<"\nname:"<<name<<"\nscore:"<<score<<"\n\n";}
protected:int num;          string name;          float score;          };
class Graduate:public Student
{public: Graduate(int n,string nam,float s,float p):Student(n,nam,s),pay(p)
    {}
    void display()
    {cout<<"num:"<<num<<"\nname:"<<name<<"\nscore:"<<score<<"\npay="<<pay<<endl;}
private: float pay;
};
void main()
{ Student stud1(1001,"Li",87.5);
  Graduate grad1(2001,"Wang",98.5,563.5);
  Student *pt=&stud1;
  pt->display();
  pt=&grad1;
  pt->display();
}
```

三、虚函数

分析：

virtual指示编译器，函数调用`pt->display()`要在运行时确定所要调用的函数，即要对该调用进行动态联编。因此，程序在运行时根据指针`pt`所指向的实际对象，调用该对象的成员函数。

实现在基类定义派生类所拥有的通用接口，而在派生类定义具体的实现方法，即常说的“**同一接口，多种方法**”，实现越来越复杂的程序。



三、虚函数

```
#include<iostream.h>
class Figure
{ public:
    Figure(double i, double j) {x=i; y=j;}
    double Area() { return 0.0;}
private:
    double x, y;
};
class Rectangle:public Figure
{ public:
    Rectangle(double i, double j, double k, double l);
    double Area() {return w*h;}
private:
    double w,h;
};
```



三、虚函数

```
Rectangle::Rectangle(double i, double j, double k, double l) :Figure(i,j)
{
    w=k; h=l; }
```

```
void fun(Figure &s)
{
    cout<<"Area="<<s.Area()<<endl; }
```

```
void main()
{
    Rectangle rec(3.0, 5.2, 15.0, 25.0);
    fun(rec);
}
```

请分析程序运行结果为什么是这样的？



三、虚函数

```
#include<iostream.h>
class Figure
{ public:
    Figure(double i, double j) {x=i; y=j;}
    virtual double Area() { return 0.0;}
private:
    double x, y;
};
class Rectangle:public Figure
{ public:
    Rectangle(double i, double j, double k, double l);
    double Area() {return w*h;}
private:
    double w,h;
};
```



三、虚函数

```
Rectangle::Rectangle(double i, double j, double k, double l) :Figure(i,j)
{
    w=k; h=l; }
```

```
void fun(Figure &s)
{
    cout<<"Area="<<s.Area()<<endl; }
```

```
void main()
{
    Rectangle rec(3.0, 5.2, 15.0, 25.0);
    fun(rec); }
```

请分析程序运行结果为什么是这样的？



五、纯虚函数

```
#include<iostream.h>
class Figure
{ public: Figure(double a,double b) { x=a;   y=b;   }
    virtual void area()
    {   cout<<"No area computation defined\n";}
protected:    double x,y;        };
class Triangle:public Figure
{public:  Triangle(double a,double b):Figure(a,b){   }
    void area()
    {   cout<<"Triangle : "<<x*y*0.5<<endl;}}
class Square:public Figure
{public:  Square(double a,double b):Figure(a,b){ }
    void area()
    {   cout<<"Square : "<<x*y<<endl;   }
class Circle:public Figure {
public:   Circle(double a):Figure(a,a) { }
    void area()
    {   cout<<"Circle: "<<x*x*3.1416<<endl;   }   };
void main()
{
    Figure *p;           Triangle t(10.0,6.0);
    Square s(10.0,6.0);   Circle c(10.0);
    p=&t;   p->area();
    p=&s;   p->area();
    p=&c;   p->area();    }
```



五、纯虚函数和抽象类

Figure是一个基类，表示具有封闭图形的东西

从Figure派生出三角形、矩形和圆，这个类等级中的基类Figure体现了一个抽象的概念，在Figure中定义一个求面积的函数显然是无意义的，

可以将其说明为虚函数，为派生类提供一个公共的接口，各派生类根据所表示的图形的不同重定义这些虚函数，以提供求面积的各自版本。

引入纯虚函数的概念



五、纯虚函数

```
#include<iostream.h>
class Figure
{ public: Figure(double a,double b) { x=a;    y=b;    }
    virtual void area()=0;
protected:    double x,y;    };
class Triangle:public Figure
{public:  Triangle(double a,double b):Figure(a,b){    }
    void area()
    { cout<<"Triangle : "<<x*y*0.5<<endl; }
};
class Square:public Figure
{public:  Square(double a,double b):Figure(a,b){ }
    void area()
    { cout<<"Square : "<<x*y<<endl;  }
};
class Circle:public Figure {
public:  Circle(double a):Figure(a,a) { }
    void area()
    { cout<<"Circle: "<<x*x*3.1416<<endl;    }    };
void main()
{
    Figure *p;          Triangle t(10.0,6.0);
    Square s(10.0,6.0);  Circle c(10.0);
    p=&t;    p->show_area();
    p=&s;    p->show_area();
    p=&c;    p->show_area();    }
```



五、纯虚函数和抽象类

抽象类

带有纯虚函数的类称为抽象类

```
class 类名
{
    virtual 类型 函数名(参数表)=0;
                                     //纯虚函数
    ...
}
```



五、纯虚函数和抽象类

抽象类

作用

- ◆对于暂时无法实现的函数，可以声明为纯虚函数，留给派生类去实现
- ◆抽象类为抽象和设计的目的而声明，将有关的数据和行为组织在一个继承层次结构中，保证派生类具有要求的行为

注意

- ◆抽象类只能作为基类来使用
- ◆不能声明抽象类的对象
- ◆构造函数不能是虚函数，析构函数可以是虚函数



五、纯虚函数和抽象类

```
#include <iostream.h>
class B0
{ public:
    virtual void display( )=0;    //纯虚函数成员
};
class B1: public B0
{ public:
    void display()
    {cout<<"B1::display()"<<endl;}
};
class D1: public B1
{ public:
    void display()
    {cout<<"D1::display()"<<endl;}
};
```



五、纯虚函数和抽象类

```
void fun(B0 *ptr)
{    ptr->display(); }
```

```
void main()
{    B0 *p;
      B1 b1;
      D1 d1;
      p=&b1;
      fun(p);
      p=&d1;
      fun(p);
}
```



三、虚函数

- 虚函数是动态绑定的基础
- 是非静态的成员函数
- 在类的声明中，在函数原型之前写**virtual**
- virtual** 只用来说明类声明中的**原型**，不能用在函数实现时
- 具有**继承性**，基类中声明虚函数，派生类中无论是否说明，同原型函数都自动为虚函数
- 本质：不是重载声明而是**覆盖**。
- 调用方式：**通过基类指针或引用，执行时会根据指针指向的对象的类，决定调用哪个函数**



三、虚函数

```
#include<iostream.h>

class Grandam
{public:
    virtual void introduce_self()
    {   cout<<"I am grandam."<<endl;   }
};

class Mother:public Grandam
{public:
    void introduce_self()
    {   cout<<"I am mother."<<endl;   }
};

class Daughter:public Mother
{public:
    void introduce_self()
    {   cout<<"I am daughter."<<endl;   }
};
```

```
void main()
{   Grandam *ptr;
    Grandam g;
    Mother m;
    Daughter d;
    ptr=&g;
    ptr->introduce_self();
    ptr=&m;
    ptr->introduce_self();
    ptr=&d;
    ptr->introduce_self(); }
```



四、虚析构函数

```
#include<iostream.h>
class Grandam
{public:
    Grandam()
    {cout<<"Grandam\n"; }
    ~Grandam()
    { cout<<"~Grandam."<<endl;  }
};
class Mother:public Grandam
{public:
    Mother()
    {cout<<"Mother\n";}
    ~Mother()
    {cout<<"~Mother()."<<endl;  }
};
void main()
{  Grandam *f;
   f=new Mother;
   delete f;}
```

请分析程序运行结果

请回答该结果是否合理？

应该如何修改该程序？



四、虚析构函数

```
#include<iostream.h>
```

```
class Grandam
```

```
{public:
```

```
    Grandam()
```

```
    {cout<<"Grandam\n"; }
```

```
    virtual ~Grandam()
```

```
    { cout<<"~Grandam."<<endl; }
```

```
};
```

```
class Mother:public Grandam
```

```
{public:
```

```
    Mother()
```

```
    {cout<<"Mother\n";}
```

```
    ~Mother()
```

```
    {cout<<"~Mother()."<<endl; }
```

```
};
```

```
void main()
```

```
{  Grandam *f;
```

```
    f=new Mother;
```

```
    delete f;}
```



小结:

1. 多态性——不同对象收到相同的消息时，产生不同的动作
2. 从实现的角度，多态可划分为两类：
 - 编译时的多态（函数重载）
 - 运行时的多态（虚函数）
3. 纯虚函数，说明在基类中，无定义，须赋初值为0。
4. 一个类至少有一个纯虚函数，则该类为抽象类。
5. 抽象类只能作为其他类的基类来使用，不能建立抽象类对象



作业：教材P229

习题3

习题5



谢 谢

